SamPr Basic

Compilatore e ambiente di sviluppo BASIC per MSX

Editor, compilatore Z80, assembler in linea e debugger — tutto sull'MSX

Manuale d'uso e riferimento del linguaggio

Versione 1.0 · MSX1 / MSX2 / MSX2+ / turbo R · MSX-DOS 1 e 2

2026 Fausto Pracek

```
PRINT "Ciao MSX!"
```

Indice

Indice 2

Presentazione 3

Installazione e avvio 4

L'editor 5

Il linguaggio in breve 11

Assembler in linea 12

Il debugger 13

Cartucce ROM, segmenti e mapper 15

Directory di lavoro (MSX-DOS 2) 16

MSX1 e MSX2 17

Riferimento del linguaggio 18

- Struttura del programma . . . 18
- Controllo del flusso . . . 20
- Dati e array . . . 24
- Testo e tastiera . . . 25
- Funzioni numeriche e stringhe . . . 27
- Schermo e grafica . . . 30
- Sprite . . . 35
- Suono . . . 37
- File su disco . . . 37
- Cartucce ROM, segmenti e mapper . . . 40
- Opzioni (OPTION) . . . 41

Errori e limiti 43

Indice delle parole chiave 44

Presentazione

SamPr Basic è un ambiente di sviluppo completo che gira **direttamente su un computer MSX**: un editor a schermo intero, un compilatore BASIC che produce vero codice macchina Z80 e un debugger, tutto senza bisogno di un PC. Si scrive il programma, si preme **F5** e dopo pochi secondi il programma gira alla velocità della macchina.

Che cosa offre

- **Compilatore vero**: ogni istruzione BASIC diventa codice Z80; il risultato è un file .COM (MSX-DOS) o una cartuccia .ROM.
- **Editor** con menu a tendina (anche da tastiera), mouse, selezione, copia/incolla, annulla/ripeti, ricerca e sostituzione, colori per le parole chiave.
- **Linguaggio strutturato**: SUB e FUNCTION con parametri, variabili locali e ricorsione; IF/ELSEIF, SELECT CASE, DO/LOOP; etichette o numeri di riga.
- **Pensato per i giochi**: grafica bitmap MSX2, sprite, accesso diretto alla VRAM, scorrimento, joystick, suono a tre voci con il linguaggio MML.
- **Assembler Z80 in linea** con `__ASM ... __ENDASM` per le parti critiche.
- **Debugger** passo-passo con breakpoint e variabili (`OPTION DEBUG`).
- **Cartucce ROM** da 16 KB a 2 MB (ASCII8, ASCII16, KONAMI, SCC) con codice su più segmenti e dati incorporati.
- **File su disco** di testo e ad accesso casuale; **mapper RAM** per grandi quantità di dati; BLOAD/BSAVE.
- **MSX1 e MSX2**: su MSX1 l'editor lavora a 40 colonne, su MSX2 a 80, scelta automatica.
- **Aiuto in linea** (F6 sulla parola sotto il cursore, F7 per l'indice) con tutte le istruzioni documentate.

Filosofia

Il linguaggio è un BASIC moderno ma volutamente piccolo: interi a 16 bit e stringhe, nessun numero in virgola mobile (che per i videogiochi costerebbe velocità e memoria senza dare vantaggi). Tutto ciò che serve a un gioco — sprite, scroll, suono, joystick, mapper — è una istruzione o una funzione. Le maiuscole e le minuscole sono equivalenti ovunque.

Installazione e avvio

Servono un MSX (MSX1 con almeno 64 KB di RAM o MSX2) con unità a dischi e MSX-DOS 1 o 2 (o Nextor). Si copiano sul disco i file seguenti:

File	Funzione
SAMPRBAS.COM	L'editor: si avvia con `SAMPRBAS NOME.SPB`
QBC.COM	Il compilatore (lanciato da F5)
RT.BIN	Il runtime del programma compilato
RTD.BIN	Runtime con debugger (per OPTION DEBUG)
QASM.OVL	L'assemblatore in linea (per __ASM)
QBCR.COM, RTROM.BIN, RTROMD.BIN	Per produrre cartucce ROM
HELP.TXT, HELP.IDX	L'aiuto in linea (facoltativi)

Avviare l'editor con `SAMPRBAS` (programma nuovo) oppure `SAMPRBAS PROVA.SPB`. Sull'MSX1 il DOS chiede la data all'avvio: premere Invio. Sul PC si può provare tutto con l'emulatore openMSX: nella cartella del progetto ci sono `avvia_nms8250.bat` (MSX2, 80 colonne) e `avvia_hb75p.bat` (Sony HB-75P MSX1, 40 colonne), che montano il disco di prova.

File dei programmi: estensione .SPB

I sorgenti hanno estensione **.SPB** (SamPr Basic). Avviando `SAMPRBAS PROVA` l'editor apre PROVA.SPB se esiste; altrimenti parte con un file nuovo che ha già quel nome (l'estensione .SPB si aggiunge da sola se manca). In **Save As** si digitano al massimo 8 caratteri e l'editor aggiunge .SPB. Il programma compilato prende lo stesso nome con estensione .COM (o .ROM per le cartucce).

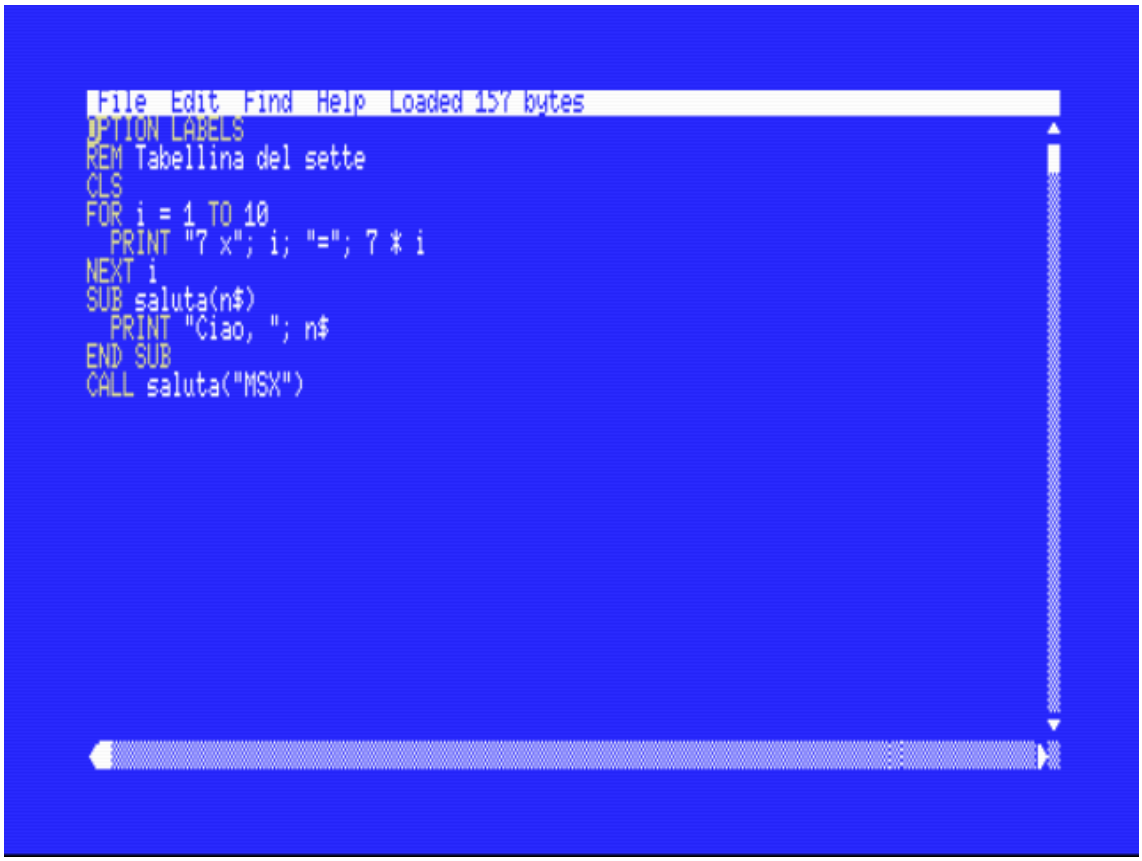
Primo programma

```
PRINT "Ciao MSX!"
FOR i = 1 TO 5
  PRINT i; " x 2 ="; i * 2
NEXT i
```

Scrivere il programma nell'editor e premere **F5**. Alla fine l'esecuzione attende un tasto e riporta all'editor. Se c'è un errore, l'editor si riapre con il cursore sulla riga sbagliata e il messaggio nella barra del titolo.

L'editor

L'editor ha una barra dei menu in alto (**File Edit Find Help Run**), l'area di testo con i numeri di riga, la barra di scorrimento e una riga di stato con posizione del cursore e nome del file. Funziona con tastiera e con mouse (se presente).



L'editor su MSX2 (80 colonne): le parole chiave sono evidenziate.

Menu da tastiera

Tasto	Azione
Esc	Apre il menu File; chiude un menu aperto
F1 F2 F3 F4	Aprono File, Edit, Find, Help
Sinistra / Destra	Con un menu aperto: passa al menu vicino
Su / Giù, Invio	Scelgono la voce
Lettera evidenziata	Attiva la voce (es. N = New, O = Open)
F5	Compila ed esegue il programma
F6	Aiuto sulla parola sotto il cursore
F7	Indice dell'aiuto

Modifica del testo

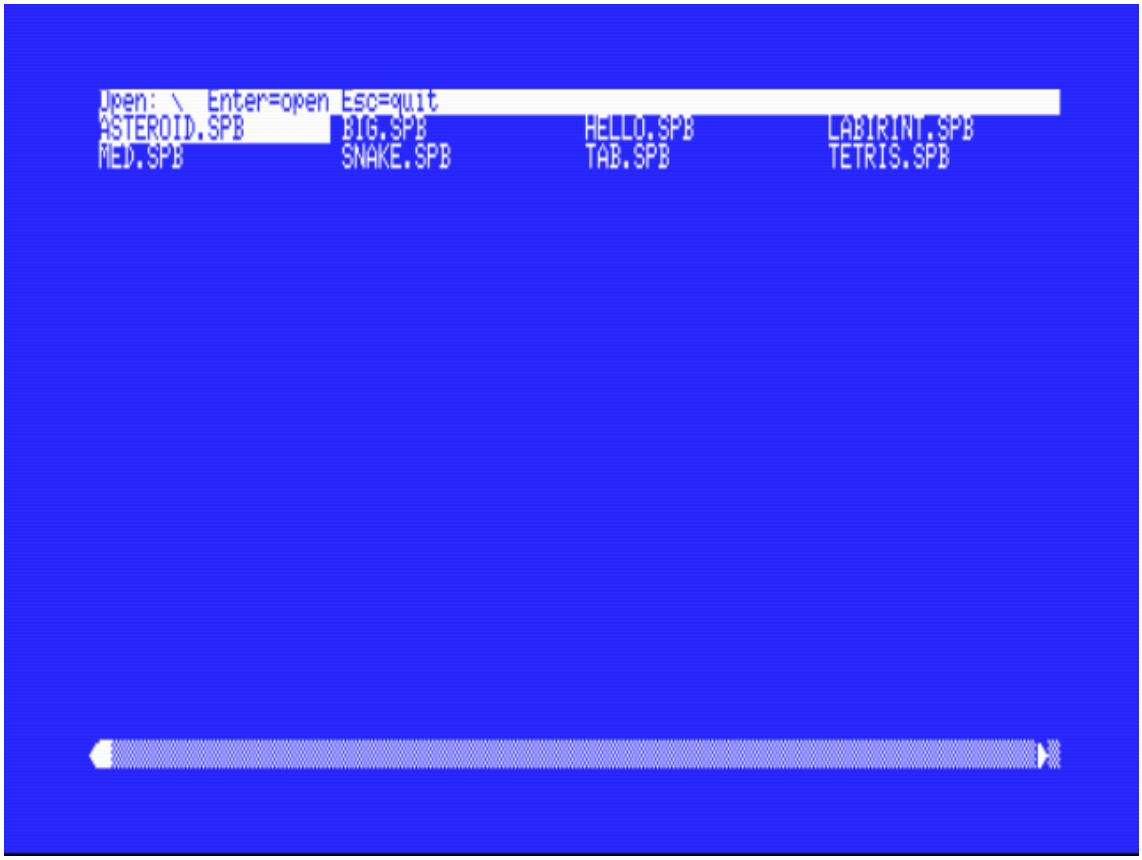
Tasto	Azione
Frecce, Home, End, PgUp, PgDn	Spostamento del cursore
Shift + frecce	Selezione
Ctrl+C / Ctrl+X / Ctrl+V	Copia, taglia, incolla
Ctrl+Z / Ctrl+Y	Annulla / ripeti
Ctrl+F / Ctrl+G / Ctrl+T	Cerca / cerca ancora / sostituisci
Ctrl+S / Ctrl+W	Salva / salva con nome
Ctrl+O / Ctrl+Q	Apri / esci

Le parole chiave del BASIC sono evidenziate in giallo (solo su MSX2). Le righe lunghe scorrono orizzontalmente. Salvando, l'editor sovrascrive il file in posizione e non salva se il testo non è cambiato; F5 senza modifiche rilancia il programma senza ricompilare.

Aprire un file: la finestra dei file

File > Open (Ctrl+O) mostra una finestra con tutti i file **.SPB** della directory corrente, in ordine alfabetico. Con MSX-DOS 2 compaiono anche le **cartelle** (fra parentesi quadre, `[GIOCHI]`, e `[..]` per salire) e si naviga fra di esse; il percorso corrente è nel titolo. Con MSX-DOS 1 non esistono cartelle e si vedono solo i file.

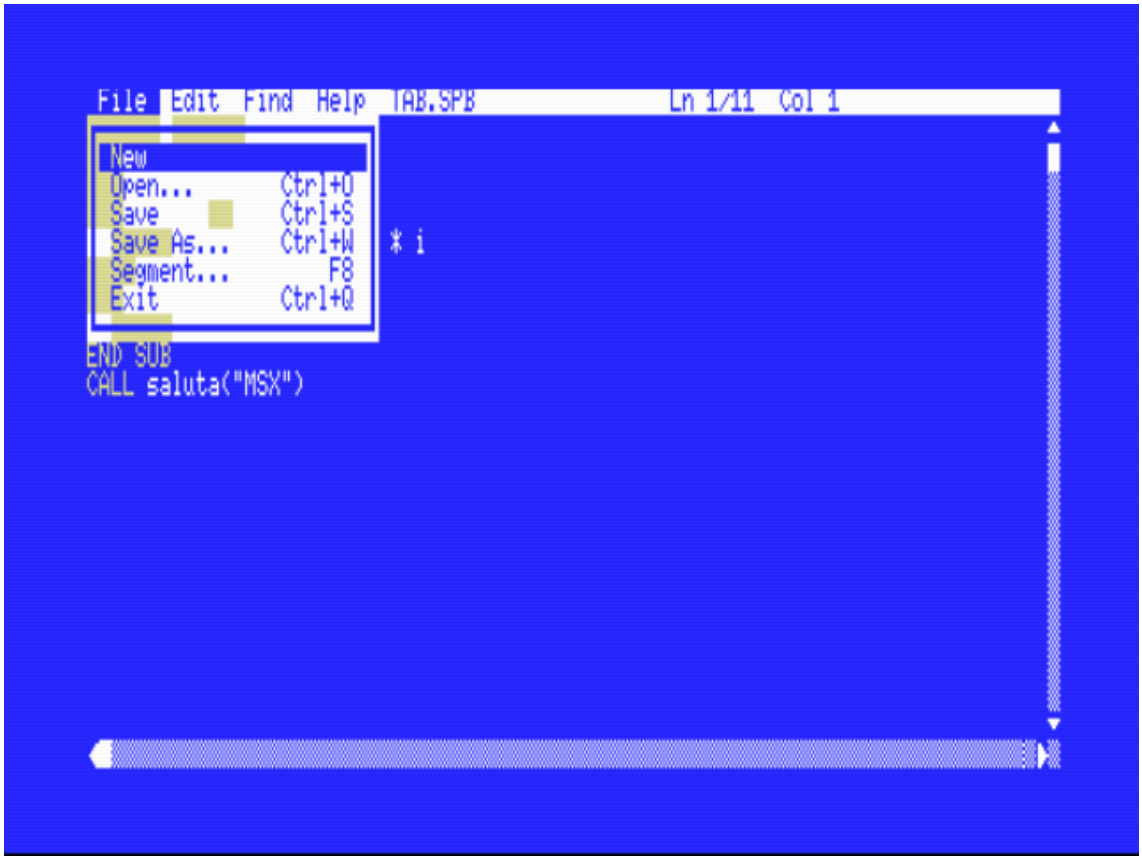
Tasto	Azione
Frecce	Spostano la selezione (due colonne su MSX1, quattro su MSX2)
PgUp / PgDn, Home	Pagina precedente / successiva, prima voce
Una lettera	Salta alla prima voce che inizia con quella lettera
Invio	Apre il file o entra nella cartella
Esc	Chiude la finestra senza aprire nulla



La finestra di apertura: tutti i file .SPB della directory, in ordine alfabetico.

Salvare con nome

File > Save As (Ctrl+W) chiede il nome: **al massimo 8 caratteri**; l'estensione .SPB è aggiunta dall'editor (se si digita un'estensione viene sostituita). Anche F5 su un programma senza nome chiede il nome nello stesso modo.



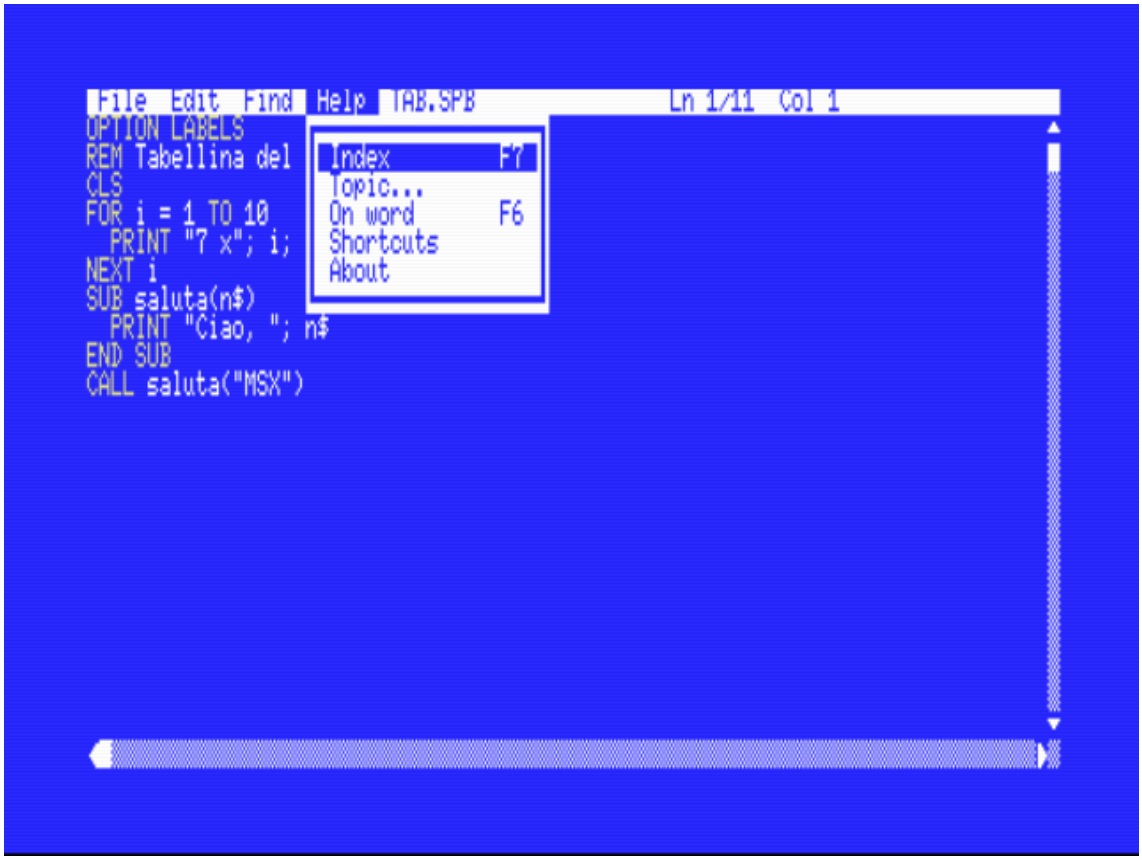
Il menu File, con la voce Segment... (F8).

Progetti a segmenti: File > Segment (F8)

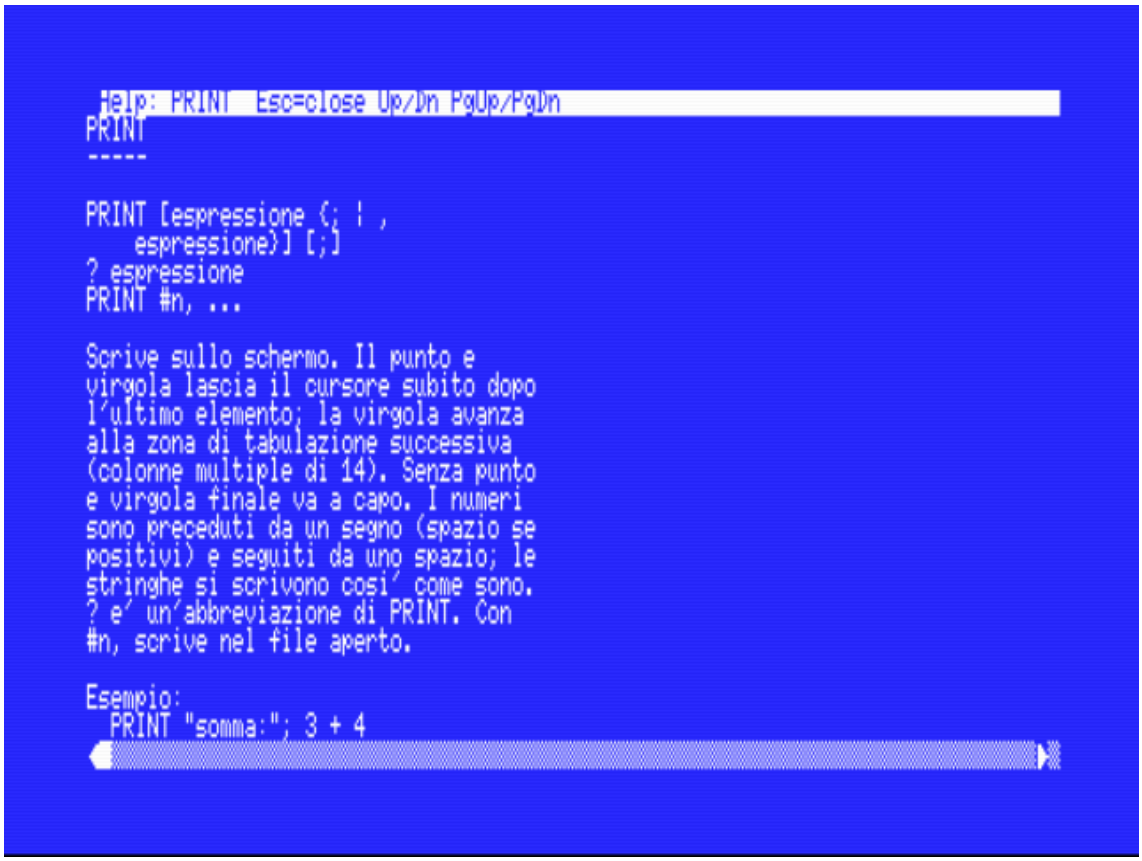
Per le cartucce a segmenti (vedi il capitolo sulle cartucce) ogni segmento è un file **NOME000.SPB**, **NOME001.SPB**, ... (5 caratteri più 3 cifre). File > Segment... (tasto **F8**) chiede il numero del segmento, anche senza zeri davanti (7 equivale a 007): l'editor **salva il file corrente**, poi cerca e apre nella stessa directory il file con quel numero; se non esiste ne crea uno nuovo già con quel nome. Il nome del file corrente deve essere di 5 caratteri più 3 cifre.

Il menu Help

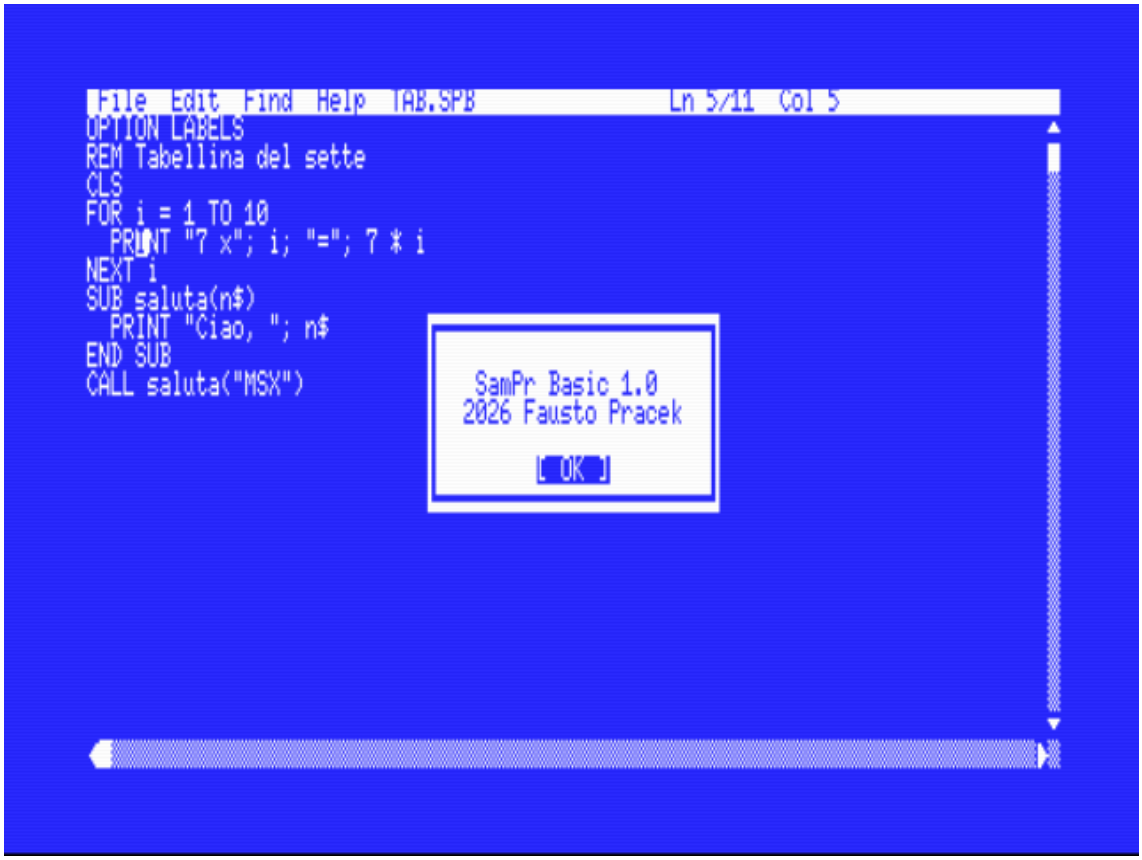
- **Index (F7)**: l'elenco di tutti gli argomenti.
- **Topic...**: chiede il nome di un argomento (PRINT, ASM, DEBUG, OPTION...).
- **On word (F6)**: documenta la parola sotto il cursore.
- **Shortcuts**: riepilogo dei tasti.
- **About**: finestra centrata con la cornice dei menu, *SamPr Basic 1.0 — 2026 Fausto Pracek* e il tasto OK; si chiude con Invio, Esc o un clic su OK.



Il menu Help.



L'aiuto sull'istruzione PRINT.



La finestra About, con la cornice dei menu e il tasto OK.

Nel visore dell'aiuto: frecce e PgUp/PgDn scorrono, Esc chiude. Il testo viene dai file HELP.TXT e HELP.IDX nella directory corrente: HELP.TXT è un normale file di testo ed è modificabile; HELP.IDX è l'indice (nome, posizione, lunghezza) rigenerato dallo strumento `doc/gen_help.py`.

Il linguaggio in breve

Un programma è una sequenza di istruzioni, una per riga (o più, separate da `:`). I commenti iniziano con `REM` o con l'apostrofo.

Tipi e variabili

Interi a 16 bit con segno e stringhe fino a 255 caratteri. Le variabili non vanno dichiarate: valgono 0 o stringa vuota. Gli array si dichiarano con DIM.

```
n = 12 : m = &HFF : b = &B101
nome$ = "Sam"
DIM v(10), t$(5)
v(3) = n * 2
t$(1) = nome$ + "!"
```

Strutture di controllo

```
IF n > 10 THEN PRINT "grande" ELSE PRINT "piccolo"

FOR i = 1 TO 10 STEP 2 : PRINT i; : NEXT i

SELECT CASE n
  CASE 1 TO 5 : PRINT "basso"
  CASE ELSE : PRINT "alto"
END SELECT
```

Routine

```
FUNCTION fatt(n)
  IF n <= 1 THEN
    fatt = 1
  ELSE
    fatt = n * fatt(n - 1)
  END IF
END FUNCTION

PRINT fatt(7)
```

Etichette o numeri

Il programma può usare etichette (`inizio:`) o numeri di riga; la direttiva OPTION sceglie il modo: NUMBERED, LABELS o MIXED (predefinita).

Assembler in linea

Per le parti che devono essere velocissime si può scrivere direttamente in assembler Z80, fra le righe **__ASM** e **__ENDASM**. Il compilatore carica un piccolo assemblatore (QASM.OVL) che traduce il blocco in codice macchina inserito nel programma.

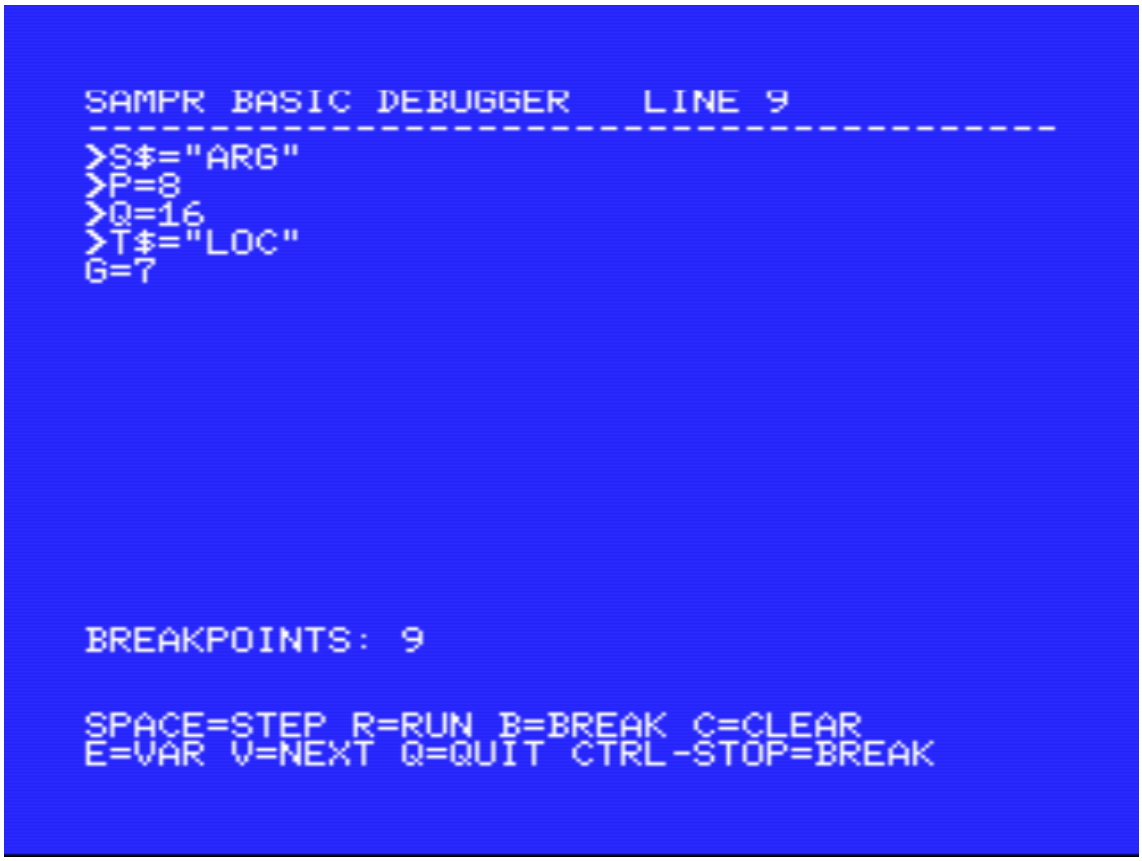
```
__ASM
  ld hl,(conta)    ; la variabile BASIC "conta"
  inc hl
  ld (conta),hl
  jp fine          ; salta all'etichetta BASIC "fine"
__ENDASM
```

Regole

- Tutte le istruzioni Z80 documentate; commenti con `;`.
- Numeri: `123, $7F, 0x7F, 7FH, %101, 101B, 'a'`; `$` è l'indirizzo corrente.
- Direttive: DB/DEFB, DW/DEFW, DS/DEFS, EQU.
- Un nome non definito nel blocco è una **variabile BASIC globale** (anche stringa) o, nei salti, un'**etichetta BASIC**.
- JR e DJNZ solo verso etichette locali.
- Si possono modificare tutti i registri tranne **IX** (frame delle routine); lo stack va lasciato in pari.
- Il debugger non si ferma dentro i blocchi. Funziona anche nelle cartucce.
- Errori 17-20: sintassi, valore fuori campo, simbolo doppio / __ENDASM mancante, QASM.OVL non trovato.

Il debugger

Aggiungere `OPTION DEBUG` in testa al programma e premere F5. Il compilatore usa il runtime con debugger: il programma parte **fermo sulla prima istruzione** e si controlla con la tastiera. Un programma senza OPTION DEBUG non paga nulla.



Il debugger fermo dentro una SUB: parametri e locali (con >), poi le globali.

Tasto	Azione
Spazio, S, Invio	Esegue una istruzione
R	Esegue fino al prossimo breakpoint
B numero Invio	Mette o toglie un breakpoint (solo Invio = riga corrente; 0 = li toglie tutti)
C	Toglie tutti i breakpoint
E	Chiede il nome di una variabile (anche locale, parametro, elemento di array come V(3), stringa come N$) e ne mostra il valore
V	Pagina successiva delle variabili
Q	Termina il programma
Ctrl-Stop	Durante l'esecuzione, rientra nel debugger

La schermata mostra la riga corrente, le variabili con il loro valore e l'elenco dei breakpoint. Dentro una SUB o FUNCTION compaiono per prime le **variabili locali e i parametri**, contrassegnati da >, poi le variabili globali scalari. Per qualsiasi altra variabile si usa il comando **E**: si digita il nome (cerca prima fra le locali, poi fra le globali); per un array si scrive l'elemento, ad esempio `V(2)` o `N$(1)`, e senza indice si ottiene il numero di elementi. Si segnalano nome inesistente, indice fuori dai limiti e array nel mapper. Lo

schermo del programma viene salvato e ripristinato: il debug funziona anche negli schermi grafici. Nelle cartucce il debugger gira ma non mostra le variabili.

Cartucce ROM, segmenti e mapper

Con `OPTION TARGET` come prima riga il compilatore produce una cartuccia **NOME.ROM** al posto del .COM: ROM16, ROM32, ASCII8, ASCII16, KONAMI, SCC. A fine programma si attende un tasto e la macchina si riavvia. Nelle cartucce non ci sono file né BLOAD/BSAVE.

Con i mapper a banchi il codice può occupare più **segmenti**: `SEGMENT n` sposta il codice che segue nel segmento n. Le routine di altri segmenti si chiamano normalmente (il compilatore cambia banco e lo ripristina). I dati si includono con `INCBIN "file"` e si leggono con `SETBANK` e `DATAWIN`.

Una cartuccia su più file

Un programma grande si scrive in **un file per segmento**. Con una OPTION TARGET a segmenti (ASCII8, ASCII16, KONAMI, SCC) i file si chiamano `NOME000.SPB` (segmento 0, quello fisso con il runtime) e `NOME001.SPB`, `NOME002.SPB`, ... (5 caratteri di nome più tre cifre). Il numero parte da **0**: il segmento 0 è il primo blocco della cartuccia (la BIOS è un'altra ROM, non c'entra), gli altri sono i blocchi commutabili.

- Le **OPTION** (TARGET, DEBUG, LABELS...) stanno **solo nel file 000**; in un altro file danno errore 13.
- **SEGMENT** non si scrive: il segmento lo decide il nome del file (con i file separati l'istruzione dà errore 13).
- Il compilatore legge tutti i file in sequenza finché trova i successivi (001, 002, ...), registra le routine di ogni segmento e le **collega in un'unica ROM** NOME.ROM (senza le tre cifre).
- Si può premere F5 da **qualunque** file del progetto: il compilatore parte sempre dal 000. Gli errori riportano il file e la riga giusti e l'editor apre quel file.
- Una SUB o FUNCTION di un altro segmento si chiama come al solito; GOTO e GOSUB non attraversano i segmenti; ogni file arriva a circa 15 KB di sorgente.
- Si passa da un file all'altro con File > Segment (F8).

Un programma piccolo resta in un solo file e può ancora usare l'istruzione `SEGMENT n`. I file senza le tre cifre finali non sono segmenti.

Il **mapper RAM** (se presente) offre segmenti da 16 KB: MAPPERSEGS, MALLOC, MFREE, MGET, MPUT, MREAD, MWRITE, VARPTR e gli array `DIM x(n) MAPPER` fino a 32768 interi.

Directory di lavoro (MSX-DOS 2)

Con MSX-DOS 2 (e Nextor) i programmi stanno in cartelle. La regola è semplice: **la directory di lavoro è sempre quella del file aperto**, per i programmi normali e per le cartucce. Il sorgente, il programma compilato (.COM, .ROM) e i file letti o scritti dal programma vivono nella cartella del progetto; gli strumenti (SAMPRBAS.COM, QBC.COM, RT.BIN, QASM.OVL...) restano nella loro cartella e vengono trovati da soli.

- Dalla riga di comando si può dare un percorso: `SAMPRBAS \GIOCHI\PROVA` entra in GIOCHI e apre PROVA.SPB (o lo crea).
- File > Open mostra le cartelle e vi si naviga; aprendo un file la directory di lavoro è quella del file.
- F5 compila e fa girare il programma nella cartella del progetto, e al ritorno l'editor è ancora lì.
- Le due directory sono memorizzate nelle variabili d'ambiente **SPBHOME** (strumenti) e **SPBPROJ** (progetto). SPBHOME vale la directory da cui si avvia l'editor la prima volta: conviene avviarlo dalla cartella degli strumenti, oppure impostarla prima con `SET SPBHOME=\STRUMENTI`.
- Con MSX-DOS 1 non esistono cartelle e nulla di tutto questo si applica.

MSX1 e MSX2

Funzione	MSX1	MSX2
Editor	40 colonne, senza colori	80 colonne, parole chiave colorate
SCREEN	0-3	0-8 (10-12 su MSX2+)
Grafica bitmap (PSET, LINE, CIRCLE, PAINT, GET/PUT)	SCREEN 2-3	SCREEN 3, 5-8
PALETTE, COPY, SET PAGE, SET SCROLL, VBANK	no	sì
Sprite	sì	sì
Suono (PLAY, SOUND, BEEP)	sì	sì
Mapper RAM	no	se presente

Il sistema riconosce la macchina da solo. Provato con Sony HB-75P (MSX1) e MSX2 con MSX-DOS 1 e 2.

Riferimento del linguaggio

Tutte le istruzioni e funzioni, raggruppate per categoria. Le sintassi usano questa convenzione: [ ] = facoltativo, { } = ripetibile, | = alternativa. Il nome di una funzione o istruzione si scrive indifferentemente in maiuscolo o minuscolo.

Struttura del programma

REM

```
REM commento
' commento
```

Commento: tutto il resto della riga viene ignorato. L'apostrofo si può mettere anche dopo un'istruzione, sulla stessa riga.

```
REM questo e' un commento
PRINT "ciao"   ' anche questo
```

Vedi anche: PRINT

ETICHETTE

```
nome:
10 PRINT "x"
```

Un programma può usare etichette (`nome:` da solo su una riga o davanti a un'istruzione) e/o numeri di riga. GOTO, GOSUB e il ripristino con RESTORE accettano l'una o l'altro. Il modo si sceglie con OPTION: NUMBERED (solo numeri, ordinati per numero), LABELS (solo etichette) o MIXED (predefinito, entrambi). I nomi delle etichette seguono le regole dei nomi di variabile. Più istruzioni sulla stessa riga si separano con i due punti.

```
inizio:
  PRINT "a" : PRINT "b"
  GOTO fine
  PRINT "mai"
fine:
  PRINT "fine"
```

Vedi anche: OPTION, GOTO, GOSUB

TIPI

```
a   a$   a(10)   a$(10)
```

Esistono due tipi: interi a 16 bit con segno (da -32768 a 32767) e stringhe (fino a 255 caratteri). Il nome di una variabile stringa termina con `$`; `A` e `A$` sono variabili distinte. Il suffisso `%` si accetta e si ignora (`A%` è `A`). I nomi non distinguono maiuscole e minuscole; sono significativi i primi 11 caratteri più il suffisso. Le variabili non dichiarate valgono 0 o la stringa vuota. Valori numerici: decimali, esadecimali `&H1F`, binari `&B101`. Il valore "vero" di un confronto è -1, il "falso" 0.

```
n = &HFF
b = &B1010
PRINT n; b
PRINT 3 > 2; 3 < 2
```

Vedi anche: DIM, CONST, OPERATORI

OPERATORI

```
+  -  *  /  \  MOD
=  <>  <  >  <=  >=
NOT  AND  OR  XOR
```

Aritmetici: `+ - * /` e `\` (divisione intera, anche `/` fra interi) e `MOD` (resto). Confronti: `= <> < > <= >=` (anche fra stringhe, in ordine alfabetico). Logici bit a bit sugli interi: `NOT AND OR XOR`. Il `+` concatena le stringhe. In caso di dubbio sulla precedenza usare le parentesi: prima gli operatori aritmetici, poi i confronti, poi quelli logici.

```
PRINT 7 \ 2; 7 MOD 2
PRINT 5 AND 3; 5 OR 3; 5 XOR 3
PRINT "ab" + "cd"; "a" < "b"
```

Vedi anche: TIPI

CONST

```
CONST nome = [-]numero {, nome = numero}
```

Definisce costanti numeriche. Si usano nelle espressioni e come dimensione di DIM; assegnare un valore a una costante è un errore.

```
CONST max = 5, base = 100
DIM a(max)
a(max) = base + 1
PRINT a(5)
```

Vedi anche: DIM

LET

```
[LET] variabile = espressione
[LET] array(i) = espressione
```

Assegna un valore a una variabile o a un elemento di array. La parola LET è facoltativa. Il tipo dell'espressione deve coincidere con quello della variabile (altrimenti errore 14: tipi incompatibili).

```
LET a = 5
b$ = "ciao"
c = a * 2
PRINT b$; c
```

Vedi anche: DIM

END

```
END
STOP
```

Termina il programma. In un programma .COM mostra "Press any key.." e attende un tasto (poi torna all'editor); in una cartuccia attende un tasto e riavvia la macchina. Alla fine del sorgente il programma termina da solo. In un programma STOP equivale a END (non c'è il debugger interattivo: per questo vedi OPTION DEBUG). Con un secondo termine END chiude anche i blocchi: END IF, END SUB, END FUNCTION, END SELECT.

```
PRINT "inizio"
END
PRINT "mai"
```

Vedi anche: DEBUG

DECLARE

```
DECLARE SUB nome(...)
DECLARE FUNCTION nome(...)
```

Accettata per compatibilità e ignorata: le firme di SUB e FUNCTION si leggono con una pre-scansione del sorgente, quindi una routine si può chiamare prima di essere definita.

Vedi anche: SUB, FUNCTION

Controllo del flusso

IF

```
IF condizione THEN istruzione [ELSE istruzione]
IF condizione THEN
  ...
[ELSEIF condizione THEN
  ...]
[ELSE
  ...]
END IF
```

Esegue istruzioni in base a una condizione (vera se diversa da zero). Forma su una riga: `IF a > 3 THEN PRINT "si" ELSE PRINT "no"`. Forma a blocchi: `IF ... THEN` a fine riga, un numero qualsiasi di `ELSEIF`, un `ELSE` facoltativo e `END IF`. I blocchi si possono annidare.

```
n = 5
IF n > 9 THEN
  PRINT "grande"
ELSEIF n > 3 THEN
  PRINT "medio"
ELSE
  PRINT "piccolo"
END IF
```

Vedi anche: SELECT CASE, OPERATORI

FOR

```
FOR variabile = inizio TO fine [STEP passo]
  ...
NEXT [variabile]
```

Ciclo con contatore. STEP è una costante intera, anche negativa; se omesso vale 1. Il ciclo non viene eseguito se il valore iniziale supera già il finale. `EXIT FOR` esce dal ciclo. I cicli si possono annidare.

```
FOR i = 1 TO 10 STEP 3
  PRINT i;
NEXT i
PRINT
FOR j = 3 TO 1 STEP -1
  PRINT j;
NEXT j
```

Vedi anche: EXIT

WHILE

```
WHILE condizione
  ...
WEND
```

Ripete il blocco finché la condizione è vera (controllo all'inizio). `EXIT WHILE` esce dal ciclo.

```
n = 3
WHILE n > 0
  PRINT n;
  n = n - 1
WEND
```

Vedi anche: DO, EXIT

DO

```
DO [WHILE|UNTIL condizione]
  ...
LOOP [WHILE|UNTIL condizione]
```

Ciclo con controllo all'inizio (condizione dopo DO) o alla fine (dopo LOOP), oppure senza condizione (ciclo infinito, da interrompere con `EXIT DO`). UNTIL ripete fino a quando la condizione diventa vera.

```
n = 0
DO
  n = n + 1
LOOP UNTIL n >= 4
PRINT n
```

Vedi anche: WHILE, EXIT

EXIT

```
EXIT FOR | EXIT WHILE | EXIT DO
EXIT SUB | EXIT FUNCTION
```

Esce subito dal ciclo (FOR, WHILE, DO) o dalla routine (SUB, FUNCTION) in corso.

```
FOR i = 1 TO 100
  IF i * i > 50 THEN EXIT FOR
NEXT i
PRINT i
```

Vedi anche: FOR, DO

SELECT CASE

```
SELECT CASE espressione
  CASE valore {, valore | da TO a}
  CASE IS operatore valore
  CASE ELSE
END SELECT
```

Sceglie fra più alternative in base a un valore intero o a una stringa. Un CASE può elencare valori (`CASE 2, 3`), intervalli (`CASE 4 TO 6`), confronti (`CASE IS >= 7`); con le stringhe anche `CASE "a", "b" TO "m"`. Il tipo dei valori deve coincidere con quello del selettore. `CASE ELSE` raccoglie il resto.

```
n = 5
SELECT CASE n
  CASE 1, 2
    PRINT "poco"
  CASE 3 TO 6
    PRINT "medio"
  CASE IS > 6
    PRINT "tanto"
  CASE ELSE
    PRINT "zero"
END SELECT
```

Vedi anche: IF

GOTO

```
GOTO etichetta | numero
```

Salta all'etichetta o al numero di riga indicato. In una cartuccia a più segmenti la destinazione deve stare nello stesso segmento.

```
i = 0
giro:
  i = i + 1
  PRINT i;
  IF i < 3 THEN GOTO giro
```

Vedi anche: GOSUB, ETICHETTE

GOSUB

```
GOSUB etichetta | numero
RETURN
```

GOSUB chiama una subroutine che inizia all'etichetta (o al numero di riga) e RETURN torna all'istruzione dopo la chiamata. Per routine con parametri e variabili locali usare SUB e FUNCTION.

```
GOSUB saluta
PRINT "fine"
END
saluta:
  PRINT "ciao"
  RETURN
```

Vedi anche: SUB, CALL

SUB

```
SUB nome [(parametro {, parametro})]
  ...
END SUB
```

Definisce una procedura. I parametri si passano per valore (interi o stringhe, con `$`); un array si passa per riferimento scrivendo `a()`. Le variabili usate dentro la SUB sono locali, salvo quelle dichiarate con SHARED. Si può chiamare con CALL o direttamente per nome. La ricorsione è consentita. Massimo 20 routine, 8 parametri ciascuna.

```
SUB somma(a, b)
  PRINT a + b
END SUB
CALL somma(3, 4)
somma 5, 6
```

Vedi anche: FUNCTION, CALL, SHARED

FUNCTION

```
FUNCTION nome[$] [(parametri)]
  ...
  nome = valore
END FUNCTION
```

Come una SUB ma restituisce un valore: si assegna al nome della funzione (`nome = ...` oppure `nome$ = ...` per le stringhe). Si usa dentro le espressioni. Ricorsione consentita.

```
FUNCTION doppio(n)
  doppio = n * 2
END FUNCTION
PRINT doppio(21)
```

Vedi anche: SUB, CALL

CALL

`CALL nome [(argomenti)]`
`CALL FAR nome [(argomenti)]`

Chiama una SUB. Le parentesi sono facoltative. `CALL FAR` forza la chiamata fra segmenti di una cartuccia (di norma il compilatore la riconosce da solo). CALL chiama una SUB, non un'etichetta: per le etichette usare GOSUB.

```
SUB p(x$)
  PRINT x$
END SUB
CALL p("ciao")
```

Vedi anche: SUB, SEGMENT

SHARED

`SHARED variabile {, variabile}`

Dentro una SUB o FUNCTION dichiara che i nomi indicati sono le variabili globali (e non variabili locali con lo stesso nome).

```
totale = 0
SUB aggiungi(n)
  SHARED totale
  totale = totale + n
END SUB
aggiungi 5
aggiungi 7
PRINT totale
```

Vedi anche: SUB, DIM

Dati e array

DIM

`DIM nome(n) [, nome$(m) ...]`
`DIM nome(n) MAPPER`

Dichiara array di interi o di stringhe con indici da 0 a n (quindi n+1 elementi). Gli indici sono controllati: fuori limite il programma si ferma con un errore. `UBOUND(a)` dà l'ultimo indice. Gli array si passano a SUB e FUNCTION scrivendo `a()`. Con MAPPER (solo `.COM` e cartucce con mapper RAM) un array di interi sta nel mapper RAM, fino a 32768 elementi.

```
DIM a(4), n$(2)
FOR i = 0 TO 4
  a(i) = i * i
NEXT i
n$(1) = "uno"
PRINT a(3); n$(1); UBOUND(a)
```

Vedi anche: UBOUND, MALLOC

UBOUND

UBOUND(array)

Restituisce il massimo indice di un array (anche quando è un parametro di una routine).

```
DIM v(9)
PRINT UBOUND(v)
```

Vedi anche: DIM

DATA

DATA valore {, valore}
READ variabile {, variabile}
RESTORE [etichetta]

DATA contiene valori (interi o stringhe fra virgolette); READ li legge in ordine nelle variabili indicate; RESTORE riporta la lettura al primo DATA oppure a quello che segue immediatamente l'etichetta indicata. Leggere oltre l'ultimo DATA è un errore.

```
DATA 10, -20, "ciao"
READ a, b, s$
PRINT a; b; s$
RESTORE
READ c
PRINT c
```

Vedi anche: DIM

MKI$

MKI$(n)
CVI(stringa$)

MKI$ converte un intero in una stringa di 2 byte; CVI fa l'operazione inversa. Servono con i file ad accesso casuale (campi FIELD).

```
a$ = MKI$(1000)
PRINT LEN(a$); CVI(a$)
```

Vedi anche: FIELD

Testo e tastiera

PRINT

```
PRINT [espressione {; | , espressione}] [;]
? espressione
PRINT #n, ...
```

Scrive sullo schermo. Il punto e virgola lascia il cursore subito dopo l'ultimo elemento; la virgola avanza alla zona di tabulazione successiva (colonne multiple di 14). Senza punto e virgola finale va a capo. I numeri sono preceduti da un segno (spazio se positivi) e seguiti da uno spazio; le stringhe si scrivono così come sono. `?` è un'abbreviazione di PRINT. Con `#n,` scrive nel file aperto.

```
PRINT "somma:"; 3 + 4
PRINT 1, 2, 3
PRINT "a";
PRINT "b"
```

Vedi anche: LOCATE, INPUT, OPEN

INPUT

```
INPUT ["testo" ;|,] variabile
INPUT #n, variabile {, variabile}
```

Legge una riga dalla tastiera nella variabile (intera o stringa). Con il testo e il punto e virgola il testo è seguito da "? "; con la virgola no. Si conferma con Invio; Backspace cancella. Un valore non numerico in una variabile intera dà 0. Con `#n,` legge dal file.

```
INPUT "Nome"; n$
INPUT "Eta", e
PRINT "Ciao "; n$; e + 1
```

Vedi anche: LINE INPUT, INKEY$

LINE INPUT

```
LINE INPUT [#n,] variabile$
```

Legge una riga intera (spazi e virgole compresi) in una variabile stringa; da tastiera o, con `#n,`, da un file.

```
LINE INPUT a$
PRINT "[" ; a$ ; "]"
```

Vedi anche: INPUT

CLS

```
CLS
```

Cancella lo schermo e porta il cursore in alto a sinistra (nella pagina attiva in SCREEN 5-8).

```
CLS
PRINT "pulito"
```

Vedi anche: LOCATE

LOCATE

`LOCATE riga, colonna`

Posiziona il cursore del testo. Riga e colonna partono da 1.

```
CLS
LOCATE 5, 10
PRINT "qui"
```

Vedi anche: PRINT, CLS

COLOR

`COLOR primo piano[, sfondo]`
`COLOR = (n, r, g, b)`
`COLOR = RESTORE`
`COLOR SPRITE(n) = colore`
`COLOR SPRITE$(n) = colori$`

Imposta i colori del testo (e il bordo) con numeri da 0 a 15 (in SCREEN 8 il valore grezzo 0-255). `COLOR = (n,r,g,b)` ridefinisce un colore della palette (r, g, b da 0 a 7; MSX2) e `COLOR = RESTORE` ripristina la palette predefinita. COLOR SPRITE vedi gli sprite.

```
COLOR 15, 4
CLS
PRINT "bianco su blu"
```

Vedi anche: PALETTE, SCREEN

INKEY$

`INKEY$`

Restituisce il carattere del tasto premuto, oppure la stringa vuota se non ce n'è: non attende.

```
DO
  k$ = INKEY$
LOOP WHILE k$ = ""
PRINT "tasto:"; ASC(k$)
```

Vedi anche: INPUT, STICK

Funzioni numeriche e stringhe

LEN

`LEN(stringa$)`

Lunghezza di una stringa.

```
PRINT LEN("ciao")
```

Vedi anche: MID$

ASC

`ASC(stringa$)`

Codice del primo carattere della stringa (0 se è vuota).

```
PRINT ASC("A")
```

Vedi anche: CHR$

CHR$

`CHR$(codice)`

Stringa di un carattere con il codice dato (0-255).

```
PRINT CHR$(72); CHR$(105)
```

Vedi anche: ASC

VAL

`VAL(stringa$)`

Valore numerico di una stringa; si ferma al primo carattere non valido, 0 se non inizia con un numero.

```
PRINT VAL("42") + 1
```

Vedi anche: STR$

STR$

`STR$(numero)`

Rappresentazione decimale del numero come stringa (con lo spazio iniziale per i positivi).

```
a$ = STR$(7)
PRINT LEN(a$); a$
```

Vedi anche: VAL

LEFT$

`LEFT$(stringa$, n)`

I primi n caratteri della stringa.

```
PRINT LEFT$("abcdef", 3)
```

Vedi anche: RIGHT$, MID$

RIGHT$

`RIGHT$(stringa$, n)`

Gli ultimi n caratteri della stringa.

```
PRINT RIGHT$("abcdef", 2)
```

Vedi anche: LEFT$, MID$

MID$

`MID$(stringa$, inizio [, n])`

Sottostringa: parte dal carattere `inizio` (il primo è 1) e prende n caratteri (fino alla fine se manca n).

```
PRINT MID$("abcdef", 2, 3)
PRINT MID$("abcdef", 4)
```

Vedi anche: LEFT$, RIGHT$, LEN

ABS

`ABS(n)`

Valore assoluto.

```
PRINT ABS(-7)
```

Vedi anche: SGN

SGN

`SGN(n)`

Segno: -1, 0 oppure 1.

```
PRINT SGN(-9); SGN(0); SGN(4)
```

Vedi anche: ABS

RND

`RND(n)`
`RANDOMIZE seme`

RND(n) restituisce un numero pseudocasuale da 0 a n-1. RANDOMIZE imposta il seme della sequenza: lo stesso seme dà la stessa sequenza.

```
RANDOMIZE 7
FOR i = 1 TO 5
  PRINT RND(100);
NEXT i
```

Vedi anche: TIMER

TIMER

`TIMER`

Contatore dei quadri video (1/50 o 1/60 di secondo) letto da JIFFY; a 16 bit, riparte da zero.

```
t = TIMER
SLEEP 25
PRINT TIMER - t
```

Vedi anche: SLEEP

PEEK

`PEEK(indirizzo)`
`POKE indirizzo, valore`

PEEK legge un byte della memoria; POKE lo scrive. Indirizzi a 16 bit (da 0 a 65535, anche con valori negativi equivalenti).

```
POKE 50000, 77
PRINT PEEK(50000)
```

Vedi anche: VARPTR

POKE

`POKE indirizzo, valore`

Scrive un byte in memoria. Vedi PEEK.

```
POKE 50000, 12
PRINT PEEK(50000)
```

Vedi anche: PEEK

SLEEP

`SLEEP quadri`

Attende il numero di quadri video indicato (50 o 60 al secondo secondo lo standard della macchina).

```
PRINT "aspetta"
SLEEP 50
PRINT "fatto"
```

Vedi anche: TIMER

Schermo e grafica

SCREEN

`SCREEN modo [, dimensione sprite]`

Sceglie il modo video. MSX1: 0 (testo 40 col), 1 (testo a colori 32 col), 2 (grafica 256x192), 3 (blocchi 64x48). MSX2 aggiunge 4, 5 (256x212, 16 colori), 6 (512x212, 4 colori), 7 (512x212, 16 colori), 8 (256x212, 256 colori) e, su MSX2+/turbo R, 10-12 (colori YJK). Le istruzioni PSET, LINE, CIRCLE, PAINT, GET/PUT e COPY lavorano in SCREEN 5-8 (PSET, POINT, LINE e CIRCLE anche in SCREEN 3). Il secondo parametro è la dimensione degli sprite: 0 = 8x8, 1 = 8x8 ingranditi, 2 = 16x16, 3 = 16x16 ingranditi. Su MSX1 un modo oltre il 3 dà "Illegal function call".

Disponibilità: MSX1: modi 0-3; MSX2: 0-8 (10-12 su MSX2+)

```
SCREEN 5
COLOR 15, 4
CLS
PSET (10, 10), 15
LINE -(100, 80), 8
```

Vedi anche: PSET, LINE, SPRITE, SET

PSET

`PSET (x, y) [, colore [, operazione]]`
`PRESET (x, y) [, colore [, operazione]]`

Accende (PSET) o spegne (PRESET, colore di sfondo) un punto. Le coordinate fuori schermo vengono ritagliate. L'operazione logica è PSET, PRESET, AND, OR, XOR oppure la stessa preceduta da T (trasparente: il colore 0 non scrive).

Disponibilità: Solo MSX2 (SCREEN 3 compreso)

```
SCREEN 5
PSET (128, 100), 12
PSET (128, 100), 3, XOR
```

Vedi anche: POINT, LINE

POINT

`POINT(x, y)`

Colore del punto; -1 se fuori schermo.

Disponibilità: Solo MSX2

```
SCREEN 5
PSET (5, 5), 9
PRINT POINT(5, 5)
```

Vedi anche: PSET

LINE

`LINE [(x1, y1)] - (x2, y2) [, colore [, B|BF [, operazione]]]`

Traccia una linea; senza il primo punto parte dall'ultimo punto grafico. Con B disegna un rettangolo, con BF un rettangolo pieno. Se manca il colore vale quello di COLOR. Non va confusa con LINE INPUT (che legge una riga di testo).

Disponibilità: Solo MSX2 (SCREEN 3 compreso)

```
SCREEN 5
LINE (10, 10)-(100, 60), 15
LINE (20, 80)-(120, 140), 9, BF
```

Vedi anche: CIRCLE, PSET

CIRCLE

`CIRCLE (x, y), raggio [, colore]`

Disegna una circonferenza.

Disponibilità: Solo MSX2 (SCREEN 3 compreso)

```
SCREEN 5
CIRCLE (128, 106), 50, 11
```

Vedi anche: LINE, PAINT

PAINT

`PAINT (x, y) [, colore [, bordo]]`

Riempie un'area partendo dal punto indicato. Nei modi a 16 colori e nei 4 colori il bordo (colore che ferma il riempimento) va indicato se diverso dal colore di riempimento.

Disponibilità: Solo MSX2

```
SCREEN 5
CIRCLE (128, 106), 50, 4
PAINT (128, 106), 6, 4
```

Vedi anche: CIRCLE, LINE

DRAW

`DRAW "comandi"`

Disegna con un linguaggio a comandi: U D L R E F G H (muovi su, giù, sinistra, destra e diagonali di n punti), M x,y (vai a; +/- relativo), B davanti a un comando (muovi senza tracciare), N (traccia e torna), C colore, S scala (in quarti). Esempio: `DRAW "C7 R20 D10 L20 U10"`.

Disponibilità: Solo MSX2

```
SCREEN 5
PSET (100, 100)
DRAW "C8 R40 D30 L40 U30"
```

Vedi anche: LINE

GET

`GET (x1,y1)-(x2,y2), array`
`PUT (x, y), array [, operazione]`

GET copia un rettangolo dello schermo in un array di interi; PUT lo ridisegna con un'operazione (PSET, PRESET, AND, OR, XOR). In grafica il rettangolo dev'essere contenuto in un array dimensionato con DIM. Non vanno confusi con GET #n / PUT #n (file ad accesso casuale) e PUT SPRITE.

Disponibilità: Solo MSX2

```
SCREEN 5
DIM a(100)
CIRCLE (20, 20), 8, 15
GET (10, 10)-(30, 30), a
PUT (100, 100), a, PSET
```

Vedi anche: COPY, PUT SPRITE, OPEN

COPY

`COPY (x1,y1)-(x2,y2)[,pagina] TO (x3,y3)[,pagina] [, operazione]`

Copia un rettangolo di schermo (anche fra pagine) con il motore comandi del VDP: velocissimo, ideale per sprite software e scorrimenti.

Disponibilità: Solo MSX2

```
SCREEN 5
CIRCLE (30, 30), 20, 12
COPY (0, 0)-(60, 60) TO (150, 100)
```

Vedi anche: GET, SET

SET

```
SET PAGE visibile, attiva
SET SCROLL x, y
```

SET PAGE sceglie la pagina mostrata e quella su cui si disegna (SCREEN 5-8): si disegna fuori vista e poi si mostra, per animazioni senza sfarfallio. SET SCROLL scorre lo schermo in modo morbido (registri 23 e 18).

Disponibilità: Solo MSX2

```
SCREEN 5
SET PAGE 0, 1
CIRCLE (100, 100), 40, 15
SET PAGE 1, 1
```

Vedi anche: COPY, VBANK

VIEW

```
VIEW [(x1,y1)-(x2,y2)]
WINDOW [SCREEN] [(x1,y1)-(x2,y2)]
```

VIEW limita il disegno a una finestra dello schermo (senza argomenti la annulla); WINDOW ridefinisce il sistema di coordinate (WINDOW SCREEN: l'asse y cresce verso il basso).

Disponibilità: Solo MSX2

```
SCREEN 5
VIEW (50, 50)-(150, 150)
LINE (0, 0)-(255, 211), 15
VIEW
```

Vedi anche: LINE

PALETTE

```
PALETTE n, r, g, b
PALETTE n, colore
COLOR = (n, r, g, b)
COLOR = RESTORE
```

Ridefinisce un colore della tavolozza (r, g, b da 0 a 7); la forma con due numeri assegna al colore n il colore predefinito indicato. `COLOR = RESTORE` ripristina la palette di fabbrica.

Disponibilità: Solo MSX2

```
SCREEN 5
PALETTE 1, 7, 0, 0
LINE (10, 10)-(100, 60), 1, BF
```

Vedi anche: COLOR

VPOKE

`VPOKE indirizzo, valore`
`VPEEK(indirizzo)`

Scrive/legge un byte della memoria video. Con VBANK l'indirizzo arriva a 17 bit (128 KB su MSX2).

```
VPOKE 0, 65
PRINT VPEEK(0)
```

Vedi anche: VBANK, VFILL, VWRITE

VBANK

`VBANK n`

Sceglie il blocco da 16 KB della VRAM usato da VPOKE, VPEEK, VFILL, VWRITE, VREAD$, BLOAD/BSAVE ... S (R#14 su MSX2).

Disponibilità: Solo MSX2

```
VBANK 1
VPOKE 0, 5
```

Vedi anche: VPOKE

VFILL

`VFILL indirizzo, quanti, valore`

Riempie un'area di VRAM con un byte.

```
VFILL 0, 768, 32
```

Vedi anche: VWRITE

VWRITE

`VWRITE indirizzo, stringa$`

Scrive in VRAM i byte di una stringa.

```
VWRITE 0, "ciao"
```

Vedi anche: VREAD$, VCOPY

VREAD$

`VREAD$(indirizzo, quanti)`

Legge un'area di VRAM e la restituisce come stringa.

```
PRINT VREAD$(0, 4)
```

Vedi anche: VWRITE

BASE

`BASE(n)`

Indirizzo di VRAM delle tabelle del modo corrente (n come nel BASIC MSX: 0 nomi, 2 colori/motivi, ...).

```
PRINT BASE(0)
```

Vedi anche: VDP

VDP

`VDP(n)`
`VDP(n) = valore`

Legge o scrive il registro n del VDP (la lettura dà la copia in RAM del BIOS).

```
VDP(7) = 9
PRINT VDP(7)
```

Vedi anche: BASE, OUT

OUT

`OUT porta, valore`
`INP(porta)`

Scrive un byte su una porta di I/O; INP ne legge uno.

```
OUT 170, 3
```

Vedi anche: INP, VDP

INP

`INP(porta)`

Legge un byte da una porta di I/O.

```
PRINT INP(168)
```

Vedi anche: OUT

Sprite

SPRITE

SPRITE$(n) = stringa$
PUT SPRITE piano, (x, y), colore, motivo
PUT SPRITE piano
COLOR SPRITE(piano) = colore
COLLISION

Gli sprite sono 32 oggetti hardware. SPRITE$(n) = stringa$ definisce il motivo n: 8 byte per un 8x8 (32 per un 16x16), un bit per punto. PUT SPRITE piano,(x,y),colore,motivo mostra lo sprite; con le sole parole PUT SPRITE piano lo si nasconde. Le dimensioni (8x8 o 16x16, normale o ingrandito) si scelgono con il secondo parametro di SCREEN. COLOR SPRITE(n) = c cambia il colore di un piano; COLOR SPRITE$(n) = stringa$ dà un colore per riga (MSX2, sprite modo 2). COLLISION vale -1 se due sprite si sono toccati.

Disponibilità: MSX1 e MSX2 (SCREEN 1-8)

```
SCREEN 2, 0
SPRITE$(0) = CHR$(255)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(255)
PUT SPRITE 0, (100, 90), 8, 0
```

Vedi anche: SCREEN, COLLISION, STICK

PUT SPRITE

PUT SPRITE piano, (x, y), colore, motivo
PUT SPRITE piano

Mostra (o con il solo piano nasconde) uno sprite. Vedi SPRITE.

```
SCREEN 2, 0
SPRITE$(0) = CHR$(255)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(129)+CHR$(255)
PUT SPRITE 0, (50, 50), 15, 0
```

Vedi anche: SPRITE

COLLISION

COLLISION

Vale -1 se due sprite si sono sovrapposti dopo l'ultima lettura (letto dallo stato del VDP), 0 altrimenti.

```
IF COLLISION THEN PRINT "urto"
```

Vedi anche: SPRITE

STICK

STICK(n)

Direzione del joystick n (0 = tasti cursore, 1 e 2 = porte). Valori da 0 (fermo) a 8: 1 su, poi in senso orario.

```
d = STICK(0)
IF d = 3 THEN PRINT "destra"
```

Vedi anche: STRIG, INKEY$

STRIG

`STRIG(n)`

Pulsante di fuoco: n = 0 barra spaziatrice, 1 e 2 pulsanti A di joystick 1 e 2, 3 e 4 pulsanti B. Vale -1 se premuto.

```
IF STRIG(0) THEN PRINT "fuoco"
```

Vedi anche: STICK

Suono

BEEP

`BEEP`

Emette il beep di sistema.

```
BEEP
```

Vedi anche: SOUND, PLAY

SOUND

`SOUND registro, valore`

Scrive direttamente un registro (0-13) del generatore sonoro PSG AY-3-8910: toni, rumore, mixer, volumi, inviluppo.

```
SOUND 7, 62
SOUND 8, 12
SOUND 0, 254
SOUND 1, 0
```

Vedi anche: PLAY, BEEP

PLAY

`PLAY voce1$ [, voce2$ [, voce3$]]`

Suona una melodia su 1-3 voci con il linguaggio MML e attende la fine. Comandi: A-G (nota, con # o + diesis, - bemolle; seguite da un numero per la durata e da punti), R pausa, O ottava (1-8), L durata di default (1-64), T tempo (32-255), V volume (0-15), N n (nota numerica), < e > ottava giù/su, M e S (modulazione/forma d'inviluppo come nel BASIC MSX).

```
PLAY "T120 O4 L8 CDEFGAB>C"
PLAY "O3 C2", "O4 E2", "O4 G2"
```

Vedi anche: SOUND, BEEP

File su disco

OPEN

`OPEN nome$ FOR INPUT|OUTPUT|APPEND AS [#]n`
`OPEN nome$ [FOR RANDOM] AS [#]n LEN = k`

Apre un file (solo programmi .COM, non cartucce). n da 1 a 3. INPUT: lettura; OUTPUT: crea/azzera; APPEND: scrive in coda; RANDOM: file ad accesso casuale con record di k byte (1-128). I file aperti si chiudono da soli alla fine del programma.

Disponibilità: Solo .COM

```
OPEN "PROVA.TXT" FOR OUTPUT AS #1
PRINT #1, "riga uno"
CLOSE #1
OPEN "PROVA.TXT" FOR INPUT AS #1
LINE INPUT #1, a$
CLOSE #1
PRINT a$
```

Vedi anche: CLOSE, PRINT, INPUT, EOF, FIELD

CLOSE

`CLOSE [#n {, #n}]`

Chiude i file indicati (tutti, se senza argomenti).

Disponibilità: Solo .COM

```
CLOSE #1
```

Vedi anche: OPEN

WRITE

`WRITE #n, espressione {, espressione}`

Scrive valori nel file separati da virgole, con le stringhe fra virgolette.

Disponibilità: Solo .COM

```
WRITE #1, 5, "ab"
```

Vedi anche: PRINT, OPEN

EOF

`EOF(n)`

Vale -1 quando il file n è finito.

Disponibilità: Solo .COM

```
WHILE NOT EOF(1)
  LINE INPUT #1, a$
  PRINT a$
WEND
```

Vedi anche: OPEN

KILL

`KILL nome$`

Cancella un file dal disco.

Disponibilità: Solo .COM

```
KILL "PROVA.TXT"
```

Vedi anche: OPEN

FIELD

`FIELD #n, larghezza AS var$ {, larghezza AS var$}`
`LSET var$ = testo$`
`RSET var$ = testo$`

File ad accesso casuale: FIELD divide il record in campi associati a variabili stringa (fino a 8); LSET/RSET riempiono il campo allineando a sinistra o a destra (completando con spazi); PUT #n,r scrive il record r, GET #n,r lo legge (record da 1). LOF(n) dà il numero di record.

Disponibilità: Solo .COM

```
OPEN "R.DAT" AS #1 LEN = 20
FIELD #1, 10 AS n$, 10 AS c$
LSET n$ = "Mario"
LSET c$ = "Rossi"
PUT #1, 1
GET #1, 1
PRINT n$; c$
CLOSE #1
```

Vedi anche: OPEN, MKI$, LOF

LOF

`LOF(n)`

Numero di record del file ad accesso casuale n.

Disponibilità: Solo .COM

```
PRINT LOF(1)
```

Vedi anche: FIELD

BLOAD

`BSAVE nome$, inizio, lunghezza [, S]`
`BLOAD nome$ [, S] [, scostamento]`

Salva/carica un blocco di memoria nel formato binario MSX. Con S i dati vanno da/verso la VRAM (VBANK vale).

Disponibilità: Solo .COM

```
BSAVE "SCHERMO.BIN", 0, 6144, S
BLOAD "SCHERMO.BIN", S
```

Vedi anche: VBANK, OPEN

Cartucce ROM, segmenti e mapper

SEGMENT

`SEGMENT n`

In una cartuccia a più banchi sposta il codice che segue nel segmento n (precedente + 1). Il segmento 0 contiene il runtime. Una SUB/FUNCTION di un altro segmento si chiama normalmente; GOTO e GOSUB non attraversano i segmenti. Per progetti grandi ogni segmento puo' stare in un file NOME000, NOME001...: vedi SEGMENTI.

Disponibilità: Cartucce ASCII8/ASCII16/KONAMI/SCC

```
OPTION TARGET ASCII8
SEGMENT 1
SUB bis()
  PRINT "nel segmento 1"
END SUB
```

Vedi anche: OPTION, INCBIN, CALL

INCBIN

`INCBIN "file"`

Include i byte di un file nel segmento corrente (grafica, musica, tabelle).

Disponibilità: Cartucce multisegmento

```
SEGMENT 2
INCBIN "SPRITES.BIN"
```

Vedi anche: SETBANK, SEGMENT

SETBANK

`SETBANK n`
`DATAWIN`

SETBANK mappa il segmento n nella finestra dati; DATAWIN è l'indirizzo della finestra (A000h per i banchi da 8 KB, 8000h per ASCII16).

Disponibilità: Cartucce multisegmento

```
SETBANK 2
VCOPY DATAWIN, 0, 256
```

Vedi anche: INCBIN, VCOPY

VCOPY

`VCOPY indirizzo_memoria, indirizzo_vram, quanti`

Copia byte dalla memoria alla VRAM.

```
VCOPY DATAWIN, 0, 2048
```

Vedi anche: SETBANK, VWRITE

MALLOC

MAPPERSEGS
MALLOC
MFREE segmento

Mapper RAM: MAPPERSEGS dà il numero di segmenti da 16 KB (0 se manca il mapper); MALLOC ne assegna uno e ne dà il numero; MFREE lo restituisce.

Disponibilità: Con mapper RAM

```
s = MALLOC
PRINT MAPPERSEGS; s
MFREE s
```

Vedi anche: MGET, MPUT, DIM

MGET

MGET(segmento, offset)
MPUT segmento, offset, valore
MREAD segmento, offset, indirizzo, quanti
MWRITE segmento, offset, indirizzo, quanti
VARPTR(variabile)

Accesso al mapper RAM: MGET legge un byte, MPUT lo scrive, MREAD copia dal mapper in memoria, MWRITE da memoria al mapper. VARPTR dà l'indirizzo di una variabile (o di un elemento di array) da passare a MREAD/MWRITE. Gli indirizzi di memoria dei blocchi non devono cadere in 8000h-BFFFh.

Disponibilità: Con mapper RAM

```
s = MALLOC
MPUT s, 0, 123
PRINT MGET(s, 0)
MFREE s
```

Vedi anche: MALLOC, VARPTR, DIM

Opzioni (OPTION)

OPTION

OPTION NUMBERED | LABELS | MIXED
OPTION TARGET tipo
OPTION DEBUG

Direttive di compilazione, da mettere all'inizio del sorgente. NUMBERED: il programma ha numeri di riga e nessuna etichetta; LABELS: solo etichette; MIXED (predefinita): entrambe. TARGET sceglie il formato di uscita (vedi OPTION TARGET). DEBUG attiva il debugger (vedi DEBUG).

```
OPTION LABELS
PRINT "ok"
```

Vedi anche: OPTION TARGET, OPTION DEBUG, ETICHETTE

OPTION TARGET

OPTION TARGET ROM16 | ROM32 | ASCII8 | ASCII16 | KONAMI | SCC

Se è la prima riga, il compilatore produce una cartuccia NOME.ROM invece di un NOME.COM. ROM16: 16 KB; ROM32: 32 KB; ASCII8, ASCII16, KONAMI (senza SCC) e SCC: cartucce a banchi, con SEGMENT. La cartuccia al termine attende un tasto e riavvia la macchina. Nelle cartucce non ci sono file né BLOAD/BSAVE né il debugger completo; l'assembler in linea funziona.

```
OPTION TARGET ROM16
PRINT "Sono una cartuccia"
```

Vedi anche: SEGMENT, OPTION

OPTION NUMBERED

OPTION NUMBERED

Righe con numero (ordinate per numero, solo numeri, niente etichette). Circa 590 righe di programma.

```
10 PRINT "a"
20 GOTO 10
```

Vedi anche: OPTION, ETICHETTE

OPTION LABELS

OPTION LABELS

Solo etichette, nessun numero di riga.

```
OPTION LABELS
ciclo:
  PRINT "x";
  GOTO ciclo
```

Vedi anche: OPTION

OPTION MIXED

OPTION MIXED

Numeri di riga ed etichette insieme (predefinita).

```
OPTION MIXED
```

Vedi anche: OPTION

OPTION DEBUG

OPTION DEBUG

Compila con il debugger incorporato: il programma parte fermo sulla prima istruzione. Vedi DEBUG.

```
OPTION DEBUG
a = 1
b = a + 1
```

Vedi anche: DEBUG

Errori e limiti

N.	Significato
1	Errore di sintassi
2	Etichetta o numero di riga inesistente
3	Non ancora supportato
4	Carattere illegale
5	Blocco non chiuso
6	ELSE / END IF / NEXT / WEND senza blocco
7	Nome duplicato
8	Troppe variabili, etichette o blocchi
9	Programma troppo grande
10	Sorgente troppo grande
11	Errore di lettura
12	Errore di scrittura
13	Non ammesso con l'OPTION scelta
14	Tipi incompatibili o array non dichiarato
15	Dati del programma troppo grandi
16	RT.BIN non trovato
17	Assembler: sintassi o operando
18	Assembler: valore o salto fuori campo
19	Assembler: simbolo doppio o __ENDASM mancante
20	QASM.OVL non trovato

Limiti

- Sorgente: circa 12,5 KB per file (15 KB per le cartucce). Programma compilato: 16 KB; per le cartucce più segmenti, ciascuno con il suo file.
- 96 etichette; 100 variabili e array; 20 SUB/FUNCTION (8 parametri ciascuna); 24 blocchi annidati.
- Nomi significativi su 11 caratteri; stringhe di 255 caratteri; interi da -32768 a 32767.
- In OPTION NUMBERED circa 590 righe; tre file aperti contemporaneamente.

Indice delle parole chiave

' ERRORI MSX2 SELECT CASE
? ERRORS MULTIFILE SET
ABS ETICHETTE MWRITE SET PAGE
AIUTO EXIT NEXT SET SCROLL
AND F5 NOME000 SETBANK
APRI FIELD NOMI SGN
ASC FILE SPB NOT SHARED
ASM FOR NUMBERED SLEEP
ASSEMBLER FUNCTION OPEN SOUND
BASE GET OPERATORI SPB
BEEP GOSUB OPT DEBUG SPBHOME
BLOAD GOTO OPT LABELS SPBPROJ
BSAVE HELP OPTION SPRITE
CALL IF OPTION DEBUG STEP
CARTELLE INCBIN OPTION LABELS STICK
CASE INDEX OPTION MIXED STOP
CHR$ INDICE OPTION NUMBERED STR$
CIRCLE INKEY$ OPTION TARGET STRIG
CLOSE INP OR SUB
CLS INPUT OUT TARGET
COLLISION KILL PAINT TASTI
COLOR LABELS PALETTE THEN
COM LEFT$ PEEK TIMER
COMPATIBILITA LEN PLAY TIPI
COMPILAZIONE LET POINT TO
CONST LIMITI POKE UBOUND
COPY LIMITS PRESET UNTIL
CVI LINE PRINT VAL
DATA LINE INPUT PSET VARIABILI
DATAWIN LINEE PUT VARPTR
DEBUG LOCATE PUT SPRITE VBANK
DEBUGGER LOF RANDOMIZE VCOPY
DECLARE LOOP READ VDP
DIM LSET REM VFILL
DIRECTORY MALLOC RESTORE VIEW
DO MAPPERSEGS RETURN VPEEK
DRAW MENU RIGHT$ VPOKE
EDITOR MFREE RND VREAD$
ELSE MGET ROM VWRITE
ELSEIF MID$ ROMFILE WEND
END MIXED RSET WHILE
END FUNCTION MKI$ SALVA WINDOW
END IF MOD SCREEN WRITE
END SELECT MPUT SEGMENT XOR
END SUB MREAD SEGMENTI __ASM
EOF MSX1 SELECT __ENDASM